PDSA-FPGA: Simple Explanation

1. What is this project?

A **mining accelerator** that runs on a **Kria KV260** FPGA board.
It has two mining algorithms that can be **swapped at runtime** without rebooting the FPGA.

2. The Two Pieces

```
+---------------------------+       +-------------------+
|   STATIC SHELL (fixed)    |       | RECONFIGURABLE    |
|                           |       | PARTITION (RP)    |
|  - AXI-Lite CSR           |<------>|                   |
|    (register file)        |       | Either:           |
|  - Hashrate counter       |       |   - Bitcoin RM    |
|  - DPR monitoring         |       |   - Kaspa RM      |
|  - PDSA decision logic    |       |                   |
|  - Interrupts             |       | (One at a time)   |
+---------------------------+       +-------------------+
         ^
         | AXI bus
         |
+---------------------------+
|   Zynq PS (ARM CPU)       |
|  - Runs Linux (Ubuntu)    |
|  - Talks to CSR via /dev/mem
|  - Loads partial bitstreams
|    via fpgautil
+---------------------------+
```

Static Shell = Always present, never changes.
RP (Reconfigurable Partition) = A reserved area of the FPGA that can be swapped with a different design at runtime.

3. What is DPR?

DPR = **Dynamic Partial Reconfiguration**

Normally, to change what an FPGA does, you reload the entire chip. That takes seconds and everything stops.

With DPR, you only change a **small region** (the RP) while the rest keeps running. The static shell stays alive, counters keep counting, the Linux app keeps talking to the CSR.

How it works on Kria:

```
Step 1: Load full shell + Bitcoin RM
        fpgautil -b pdsa_full_sep_btc.bit.bin

Step 2: Start mining (app writes CSR registers)

Step 3: Swap to Kaspa (while shell keeps running)
        fpgautil -b pdsa_rm_sep_kaspa.bit.bin
        (Only the RP changes — takes ~100ms)
```

The PS (CPU) does the swap. The PL (FPGA) is passive.

4. What is PDSA?

PDSA = **Policy-Driven Self-Adaptation**

A simple idea: the FPGA watches its own performance and decides whether to switch algorithms.

```
flowchart LR
   A[Start: Bitcoin] --> B{Hashrate OK?}
   B -->|Yes| C[Keep Bitcoin]
   B -->|No| D{Profitability<br/>threshold?}
   D -->|Switch needed| E[Swap to Kaspa]
   E --> B
```

Three decisions:

Decision	Meaning
Continue	Keep current RM, hashrate is good
Switch PT	Hashrate below threshold → try other RM
Switch BCV	Blockchain validation says switch

The CSR exposes `csr_pt_threshold`, `csr_pt_current`, and `sts_pdsa_decision` — the Linux app reads these and decides whether to trigger a DPR swap.

5. The Flow in Pictures

Full boot:

```
Power ON → PS boots Linux → fpgautil loads full bitstream
        (shell + Bitcoin RM) → CSR is at 0xA000_0000
```

Mining (Bitcoin):

```
Linux app → writes target, midstate, job_data
      → writes CSR_START
      → FPGA hashes
      → result appears in sts_result_hash
      → interrupt fires
```

DPR (swap RM):

```
PDSA says "switch" → app writes csr_decouple=1
            → fpgautil loads Kaspa partial
            → app writes csr_decouple=0
            → app writes CSR_START
            → FPGA now runs Kaspa
```

6. What files do what

File	Purpose
`pdsa_fpga_dfx_top.sv`	Top level — wires shell to RP
`pdsa_static_shell.sv`	Static logic — CSR, counters, PDSA
`axi_lite_csr.sv`	57 registers, AXI-Lite slave
`rm_bitcoin.sv`	Bitcoin RM (SHA-256d)
`rm_kaspa.sv`	Kaspa RM (SHA-3 placeholder)
`pdsa_full_sep_btc.bit`	Full bitstream (shell + Bitcoin)
`pdsa_rm_sep_btc.bit`	Partial — swap to Bitcoin
`pdsa_rm_sep_kaspa.bit`	Partial — swap to Kaspa
`sw/pdsa_csr.c`	Linux app — talks to CSR
`sw/pdsa_miner.c`	Demo app — starts/stops mining

7. Summary

- **DPR** = swap a part of the FPGA without stopping the rest
- **PDSA** = logic that decides *when* to swap
- **PS does the swap** (`fpgautil`), PL provides the registers
- **2 partial bitstreams** needed (one per RM)
- **XSA not required** for deployment on Kria Ubuntu
- **Kaspa RM is a placeholder** — real HeftyHash not implemented

```
┌──────────────────────────────────────────┐
│  Questions?                              │
│  PDSA-FPGA — Proof of Concept            │
│  Kria KV260 (xck26-sfvc784-2LV-c)        │
│  Vivado 2025.2                           │
└──────────────────────────────────────────┘
```